1. Comparați A* cu alte tipuri de algoritmi de căutare.

A*	DFS
Cauta un drum de cost minim	Cauta un drum intre doua noduri
Relaxeaza muchiile in mod optim prin intermediul unei euristici	Inainteaza aleatoriu
A*	**BFS**
Se duce direct la nodul cu functia f = h + g cea mai buna	Parcurge toti vecinii, intr-o ordine aleatorie

2. Ce este un matroid și pentru ce este folosit?
Un sistem accesibil este un matroid daca satisface proprietatea de interschimbare:
VX, Y apartin I si |X| < |Y| => E e apartine Y \ X astfel incat X U {e} apartine I.
Daca (E, I) este matroid, atunci exista un algoritm Greedy care rezolva problema de optimizare.

3. Descrieți cel mai performant algoritm studiat la curs care, folosit împreună cu un algoritm de backtracking, permite reducerea complexității temporale a execuției?
Cel mai performant algoritm este AC-3 (Arc Consistency - 3). Acesta elimina pe rand arcele (constrangerile). Daca o constrangere aduce modificari in retea, adaugam pentru reverificare nodurile care puncteaza catre nodul de plecare al restrictiei verificate.
Scopul: reverificarea nodurilor direct implicate de o constrangere in retea
Avantaj: mai putine apeluri ale functiei REVISE
Complexitate: $a^3 * r$

4. Arătați de ce nu poate fi folosit algoritmul lui Dijkstra pentru grafuri cu costuri negative pe arce.
In caiet

5. De ce algoritmul Ford-Fulkerson poate fi foarte ineficient și cum îi îmbunătățește algoritmul Edmonds-Karp performanța?
Ford-Fulkerson poate fi ineficient pentru grafuri ca: … (poate trece mereu prin B-C). Complexitatea acestuia depinde de fluxul maxim (O(E * f_max)). Se folosesc cai cu capacitate mica si se folosesc mai multe arce decat este nevoie.
Imbunatatiri: Edmonds-Karp alege caile reziduale minime prin aplicarea unui BFS.

1. Cum se pot determina eficient, și cu ce complexitate temporală, componentele conexe ale unui graf neorientat?
Un algoritm eficient care poate determina componentele conexe ale unui graf neorientat este DFS. In momentul in care se termina parcurgerea dintr-un nod, inseamna ca a fost gasita o componenta conexa si se incepe urmatoarea. Complexitatea este O(V + E).

2. Cum poate fi folosit eficient algoritmul minimax în rezolvarea unor probleme parțiale într-un program de jucat șah? (împreună cu alte metode, știindu-se dimensiunea imensă a arborelui de mutări posibile) Dați două situații posibile.
Putem construi o functie euristica care sa estimeze sansele de castig pentru o pozitie data. (Ex: Regina - 10p, Turn - 5p, Cal, Nebun - 3p, Pion - 1p).
Sau putem alege intai mutarile in care se iau piese, sau mutarile cu scor bun in parcurgerile precedente.

3. Comparați algoritmii aleatori cu cei de aproximare. De ce este necesară folosirea unui algoritm aleator pentru calcularea numerelor prime?
Algoritmii aleatori gasesc solutii care fie sunt optime, fie exacte. Acestia functioneaza corect doar daca timpul de rulare este infinit. Altfel, solutiile sunt aproximative.
Pentru calcularea numerelor prime se folosesc algoritmi aleatori, deoarece o abordare clasica a problemei ar costa foarte mult timp pentru numere mari (operatiile nu mai sunt O(1)). Astfel, se

aleg numere random pentru care se testeaza fie mica teorema a lui Fermat (daca nu se respecta, numarul nu este prim), fie gasirea unui martor al divizibilitatii (Algoritmul Miller-Rabin - daca x este martor al divizibilitatii, atunci numarul nu este prim).

4. Cum se rezolvă problema costurilor negative în algoritmul lui Johnson?
Daca avem costuri negative, atunci se adauga un nod nou, s, si se traseaza arce de la S catre toate nodurile din graf, arcele avand toate costul 0. Se aplica Bellman-Ford din S si se calculează distantele minime de la S la celelalte noduri, rezultatele fiind potentialele nodurilor. Cu aceste potentiale se vor actualiza costurile muchiilor grafului initial. Noul cost al unui arc va fi w1(u, v) = w(u, v) + h(u) - h(v), unde h(u) - potentialul nodului u. Dupa aceea se aplica Dijkstra pentru toate nodurile, iar la final costurile muchiilor vor fi w(u, v) = w1(u, v) - h(u) + h(v).

5. Cum se poate modifica un algoritm studiat la curs pentru a calcula drumul critic (drumul de lungime maximă) într-un graf aciclic orientat?
??(A* - euristica coresp)

1. Menționați care sunt principalii pași pentru a rezolva o problemă de optimizare folosind tehnica de programare dinamică, precum și ce proprietăți trebuie să satisfacă problema. Exemplificați acești pași pentru problema parantezării matricilor (chain matrix multiplication problem).
O solutie optima contine solutiile optime ale subproblemelor sale.
Problema P se descompune recursiv in subprobleme care accepta solutii din ce in ce mai simple. La final are loc suprapunerea subproblemelor si aflarea solutiei optime a problemei initiale. Pe parcursul rezolvarii subproblemelor, rezultatele se retin in variabile/vectori/matrice pentru a nu se recalcula la pasii viitori.
Problema parantezarii matricelor presupune descompunerea in subprobleme. Daca solutia A(i, j) se descompune in A(i, k) si A(k + 1, j), ca A(i, j) sa fie optimala, trebuie ca si solutiile pentru A(i, k) si A(k + 1, j) sa fie optimale.

2. Detaliați ce reprezintă valoarea low calculată de către algoritmul lui Tarjan și prezentați cum este calculată această valoare pentru un nod și cum este folosită pentru identificarea punților într-un graf neorientat.
Valoarea low reprezinta timpul de debut al primului nod din ciclul in care se afla nodul (cel cu d-ul cel mai mic).
Cand se ajunge la inchiderea unui ciclu, low-ul nodului va fi minimul dintre el si timpul de debut al nodului in care ajunge (aceasta actualizare nu va avea loc daca sunt pe muchia corespunzatoare parintelui). La finalul terminarii explorarii un nod, low-ul parintelui va fi minimul dintre el si low-ul copilului.
Daca low-ul copilului este mai mare decat timpul de debut al parintelui, inseamna ca se formeaza o
punte.

3. Dați un exemplu de graf cu cel mult 5 noduri și cel puțin o muchie de cost negativ pentru care algoritmul lui Dijkstra nu calculează valoarea corectă a drumului minim de la o sursa aleasă de către voi (menționați valoarea calculată de algoritm, precum și cea corectă). Cum poate fi modificat algoritmul lui Dijkstra pentru a funcționa corect și pentru grafuri cu muchii de cost negativ și ce impact are acest lucru asupra complexității?
Desen + Algoritmul lui Dijkstra adaptat este varianta Bellman-Ford optimizat. Plecand de la algoritmul inițial al lui Dijkstra, se adauga un vector marcat unde initial este tot false. Inainte ca nodurile sa fie adaugate in coada, marcat va fi true, iar cand sunt scoase din coada, acesta devine false. Acest lucru ne ofera posibilitatea de a parcurge nodurile de mai multe ori in cazul relaxarii muchiilor.
Complexitate Dijkstra normal: O(E * logV)
Noua complexitate: O(V * E) > O(log V * E)

4. Prezentați cel puțin 3 modalități diferite de alegere a drumului de ameliorare pentru un algoritm care folosește metoda Ford-Fulkerson pentru calculul fluxului maxim. Printre acestea trebuie să fie și metoda folosită de către algoritmul Edmonds-Karp. Discutați apoi, incluzând o scurtă discuție/demonstație în propriile cuvinte, de ce algoritmul Edmonds-Karp este preferat în situații practice.

Modalitatile ar fi sa alegem:
- un drum oarecare
- o cale reziduala de capacitate maxima
- o cale reziduala de lungime minima (ceea ce foloseste Edmonds-Karp)
Vezi primul 5 de mai sus.

5. Ce este o euristică admisibilă pentru algoritmul A* și de ce este important ca euristica să fie admisibilă? Având următoarea euristică simplă: h(n) = k (k ≥ 0, constantă pozitivă fixată), n != destinație, și h(destinație) = 0, încercați să impuneți o constrângere asupra unui număr minim de muchii din graful stărilor astfel încât euristica să fie admisibilă.
O euristica admisibila este cea mai optimista. Algoritmul A* ce foloseste o euristica admisibila descopera solutia optima (daca exista).
??

1. Care sunt proprietățile pe care trebuie să le satisfacă o problemă pentru a putea fi rezolvată prin tehnica programării lacome? Exemplificați cum se aplică acestea pentru problema construcției arborilor Huffman.
Algoritmii Greedy functioneaza cand solutia problemei contine solutia subproblemelor si daca in momentul alegerii solutiei locale aceasta este de asemena si solutie globala. Pentru constructia arborilor Huffman, abordarea Greedy se foloseste cand se doreste unirea a doua frunze/mini-arbori cu ponderile si se aleg intotdeauna nodurile cu cele mai mici doua ponderi.

2. Care este semnificația valorilor d[v] pentru algoritmul lui Prim? Justificați pe scurt de ce muchiile alese de către algoritm sunt muchii sigure pentru arborele parțial minim construit până la acel moment.
Valoarea lui d[v] reprezinta muchia de cost minim ce are legatura cu nodul v si cea care imi va lega nodul cu arborele partial minim de pana la acel moment. Algoritmul fiind Greedy, alege intotdeauna muchiile de cost minim, iar solutia optim locala va deveni una globala.

3. Asemănări și deosebiri între algoritmii AC1 și AC3. Printre acestea, atingeți și aspecte legate de utilitate, funcționare, complexitate, de ce unul dintre cei doi algoritmi este mai puțin eficient și cum se poate rezolva această problemă.
AC3 face mai putine apeluri ale functiei REVISE decat AC1. La AC3, daca o constrangere se modifica, toate nodurile care puncteaza catre acel nod si doar acestea sunt adaugate pentru reverificare. La AC1, se aplica algoritmul de consistenta si revise pe toate nodurile pana nu mai am nicio modificare.
AC3 - $O(a^3 * r)$
AC1 - $O(a^3 * rn)$

4. Care este complexitatea algoritmului Edmonds-Karp? Justificați pe scurt cum se poate demonstra această limită de complexitate.
$O(E^2 * V)$
Pentru initializari: O(E)
While-ul dureaza $O(E * V)$
Gasirea caii reziduale: O(E)
Restul operatiilor din while sunt tot O(E)
=> in final $O(E + E * V * E) = O(E^2 * V)$

5. Definiți ce înseamnă admisibilitatea și dominanța pentru algoritmii de tip A*. Pentru jocul 8-puzzle discutat la curs, dați exemplu 3 euristici nebanale (care să nu fie constante, ci să depindă de pozițiile pieselor din starea curentă și din starea finală) dintre care una să nu fie admisibilă. Justificați pe scurt de ce euristicile alese sunt admisibile (sau nu), precum și care dintre cele două euristici admisibile o domină pe cealaltă.
Admisibilitatea inseamna optimism. O eurstica este admisibila daca alege intotdeauna ce o avantajeaza pentru rezolvarea problemei. Daca avem h1, h2 doua euristici admisibile, h1 domina h2 daca pentru orice nod, h1(nod) > h2(nod) (h1 ajuta mai mult algoritmul in determinarea caii celei mai mici de ex).
h1 = nr patratelor a caror pozitie difera de cea finala
h2 = suma distantelor Manhattan dintre pozitia finala si cea curenta pentru fiecare patratel
h3 = distanta minima dintre pozitia finala si cea curenta

Ultimele doua sunt admisibile, iar h3 > h2.

1. Dacă o problemă poate fi rezolvată corect atât prin greedy, cât și prin programare dinamică, ce metodă ați alege și de ce?
O problema ce se poate rezolva atat prin greedy, cat si prin dp este problema rucsacului. Daca problema ar permite impartirea greutatilor, atunci s-ar rezolva folosind greedy si sortand greutatile dupa raportul dintre pret si greutate. Daca problema ar impune o varianta discreta, atunci as folosi programarea dinamica.

2. Pornind de la definiția funcției REVISE, arătați de ce algoritmul AC3 este mai performant decât algoritmul AC1.
La AC3 avem mai putine apeluri ale functiei REVISE decat AC1.

3. Scrieți care este diferența între un flux și un preflux. Descrieți apoi pe scurt cum funcționează algoritmii de preflux pentru calculul fluxului maxim scoțând în evidența de ce la final prefluxul este un flux corect.
O functie de flux intr-o retea este o functie care respecta urmatoarele proprietati:
- are valoarea mai mica sau egala cu capacitatea muchiei;
- fluxul de la u la v este opusul fluxului de la v la u;
- fluxul care intra intr-un nod este egal cu fluxul care iese.
Diferenta dintre flux si preflux este ca fluxul care intra trebuie sa si iasa.
O analogie pentru algoritmul de pompare a prefluxului este:
Am o sticla de apa (sursa) si vreau sa pun apa intr-un pahar specific. Pe langa acestea mai avem doua pahare. Initial, sursa se afla la nivelul 4, iar paharele la 0. Torn apa din sticla in paharul/ paharele la care am legatura directa. Dupa ce paharul se umple, torn apa din pahar in pahar pana ajung in drena. Conditia este ca in momentul in care torn apa dintr-un pahar in altul, paharul din care torn trebuie sa fie cu un nivel mai sus. Dupa ce s-a umplut drena, surplusul se intoarce in sticla, urcand paharele pana la nivelul sursei + 1.
La fiecare pas, incerc sa umplu paharele la capacitatea lor, tot timpul maximizez fluxul si torn cat de mult pot. La final, stiu ca fluxul este maxim, intrucat tot ce se intoarce inapoi se intoarce pentru ca nu am mai avut loc in drena, aceasta fiind umpluta.

4. Comparați algoritmii de tip Las Vegas și Monte Carlo, scoțând în evidență asemănările și diferențele între acestea.
Ambii algoritmi sunt aleatori si reduc timpul de rezolvare a problemelor ce contin volume mari de date. Cand sunt lasati sa ruleze la infinit, ambii isi ating scopul (LV - obtine solutia optima; MC - gaseste solutia corecta).
Las Vegas trebuie sa gaseasca solutia optima cu o anumita marja de eroare, iar Monte Carlo cauta solutia corecta.

5. Ce se întâmplă cu algoritmul minimax cu tăieturi alfa-beta dacă modificăm condiția de tăiere din (β<α) în (β-k<α), unde k este o constantă oarecare. Discuție după k.
1. k = 0 => algoritmul nu se modifica
2. k < 0 => se omit anumite taieturi si va creste timpul de rulare, insa solutia finala va fi cea corecta
3. k > 0 => se fac taieturi in plus, algoritmul se termina mai rapid, insa rezultatul va fi gresit

1. Comparatie intre tehnicile de programare lacoma si programare dinamica.

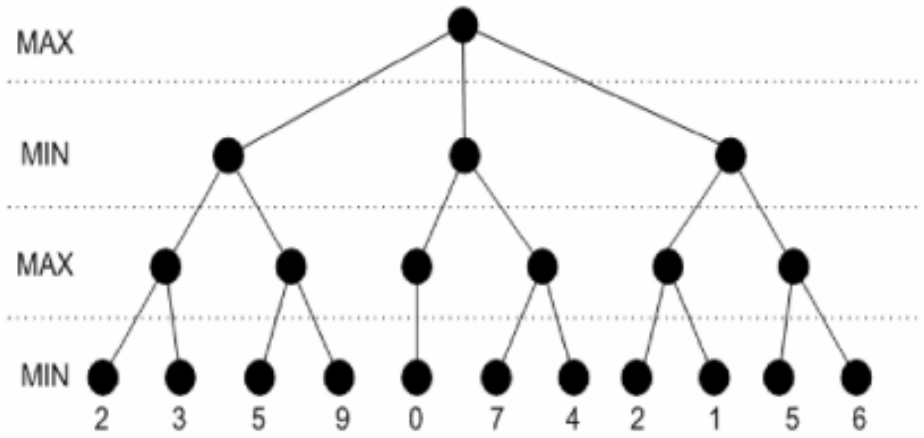
Greedy	DP
Sunt mentinute doar solutiile partiale curente care se transforma in solutiile partiale urmatoare	La constructia unei solutii noi poate contribui orice alta solutie patsiala aflata anterior
Solutiile partiale sunt elimintate odata ce s-a trecut mai departe	Se patreaza toate solutiile partiale
Se poate obtine o solutie neoptima	Se obtine solutia optima

5. Explicati principiul taieturilor alfa-beta si care este imbunatatirea adusa comparand (analiza a complexitatii) algoritmul minimax cu alfa-beta cu algoritmul minimax simplu.
Taierile alfa-beta limiteaza spatiul de cautare prin elminarea unor variante.
Alfa = max dintre valorile gasite pentru un nod MAX
Beta = min dintre valorile gasite pentru un nod MIN
Taiem o ramura daca:
- Am gasit un nod de pe nivelul MAX cu valoare beta <= orice alfa calculat anterior
- Am gasit un nod de pe nivelul MIN cu valoare alfa >= orice beta calculat anterior
Complexitate alfa-beta: $O(b^{d/2})$ (b - branching factor; d - depth)
Complexitate minimax: $O(b^d)$ > complexitate alfa-beta

1. Asemanari si deosebiri intre algoritmul lui Prim si cel al lui Kruskal.
Ambii algoritmi calculeaza arborele minim de acoperire al unui graf. Se bazeaza pe greedy, alegand mereu solutia optima (cea de cost mai mic).
Prim formeaza treptat arborele, plecand de la nodul sursa catre vecini, gasind muchiile de cost minim ce-l extind.
Kruskal cauta in intregul graf muchiile de cost minim si incepe sa lege acei mini-arbori formati doar in momentul in care gaseste o muchie ce are un capat intr-un graf si celalalt capat in al doilea graf.
In momentul in care am un ciclu si muchia de cost maxim apare de cel putin doua ori in acesta, atunci algoritmii pot returna arbori diferiti de acoperire (va conta ce muchie/nod este parcursa prima la sortare). In rest, cei doi algoritmi intorc acelasi arbore minim de acoperire.

3. O euristica monotona asigura optimalitatea algoritmului A*? Justificati raspunsul vostru (demonstratie, explicatie in propriile cuvinte, etc).
Daca euristica este monotona, inseamna ca pentru orice n si n' (succesorul lui n) h(n) <= h(n') + c(n, n'), unde c(n, n') = costul arcului de la n la n'. Aceasta euristica este una optimista, deoarece ea se intampla pentru orice pereche de noduri n si n'. (corolar 7.2 - giumale) Cum euristica admisibila determina algoritmul A* sa gaseasca solutia optima => o euristica monotona asigura optimalitatea.

5. Pentru arborele de joc din figura alaturata, evidentiati taieturile alfa-beta si valorile alese in fiecare nod.



Taieturi: 9, 4, 6
Nivel 1: 3, 5, 0, 7, 2, 5
Nivel 2: 3,
Nivel 3:

1. Explicati din ce motiv muchia cu costul cel mai mare din orice ciclu care apare intr-un graf neorientat nu poate face parte din nici un arbore minim de acoperire, indiferent de algoritmul folosit pentru determinarea AMA (pentru simplificare, puteti presupune ca nu mai exista nici o alta muchie de acelasi cost cu aceasta).
Iau un exemplu si arat ca daca as considera acea muchie in AMA, as gasi un alt AMA cu costul mai mic => nu se poate

2. Ce este reteaua reziduala a unui graf? Cum poate fi aceasta folosita pentru a calcula fluxul maxim? Justificati care este numarul minim, respectiv numarul maxim de arce care pot apare in reteaua reziduala a unui graf G(V, E).
Reteaua reziduala Gf = (V, Ef) unde Ef = {(u, v) apartine V x V | c(u, v) - f(u, v) > 0}
Gf include toate arcele pe care se poate mari fluxul.
Odata ce cunosc arcele pe unde pot mari fluxul, atunci gasesc o cale de la sursa la drena care sa poata creste fluxul prin retea.
Numarul minim este reprezentat de numarul muchiilor (E) in cazul in care ori n-am niciun flux care trece prin muchii, ori toate muchiile sunt la capacitate maxima, ori o combinatie dintre acestea doua.
Numarul maxim este 2 * E in momentul in care am atat flux intre doua noduri, cat si capacitate reziduala.

3. Pornind de la definitii (scrieti si aceste definitii), ce relatie se poate stabili intre g(n) si g*(n) pentru orice cautare A*, indiferent de problema de rezolvat si euristica folosita? Cand se poate garanta ca g(n) = g*(n) pentru orice stare descoperita/expandata de A*?
g(n) este costul drumului n_0…n descoperit de algoritm la momentul curent de timp;
g*(n) este costul exact al unui drum optim n_0…n
g*(n) <= g(n)
Cand n este n_0 (g(n) = 0) sau cand n este un nod izolat sau face parte dintr-un alt graf si nu pot ajunge la el din nodul sursa (g(n) = inf).

5. Explicati ce inseamna taierea alfa-beta intr-un arbore de joc, de ce acestea sunt utile si dati un exemplu simplu de arbore de joc unde aplicati o taiere alfa-beta (scotand in evidenta ceea ce taiati).
Explicat mai sus.
MAX: A(5)
MIN: B(5) C(2) D(4)
12 5 9 2 7 1 8 4 9
(pag16)

3. Care este algoritmul generic pentru determinarea arborelui minim de acoperire al unui graf? Cum pot fi derivati algoritmii Prim si Kruskal pornind de la acest algoritm generic?
Alogritmul generic respecta doua proprietati:
1) intr-un ciclu muchia de cost maxim nu face parte din AMA;
2) daca un nod face parte din AMA partial, atunci va fi adaugat si nodul care este legat de acesta prin muchia de cost minim.
Prim se bazeaza pe a doua prorietate, deoarece porneste de la un nod si adauga treptat nodurile care sunt legate prin muchiile de cost minim de ceea ce se afla in AMA partial pana la acel moment.
Kruskal se foloseste de prima proprietate, fiindca adauga muchii in ordinea costurilor lor (nu tine cont de legaturile dintre noduri) si cand trebuie sa uneasca doua noduri, evita ciclurile.

4. Explicati utilitatea si functionarea algoritmului lui Johnson. De ce este mai bun acest algoritm fata de alternative?
Algoritmul aplica Dijkstra pe rand pe fiecare nod din graf si afla distanțele minime intre ele. Pentru costuri negative se aplica Bellman-Ford inainte. Acesta porneste dintr-un nod nou adaugat (...). Fata de Floyd-Warshall ($O(V^3)$), acesta are o complexitate ai buna ($O(V^2 * \log V + V * E)$) pentru grafuri rare. In plus, Johnson detecteaza prin Bellman-Ford ciclurile de cost negativ si se opreste, fara a mai calcula distantele.

5. Care este complexitatea algoritmului A*? Cum este aceasta complexitate fata de complexitatea algoritmilor neinformati?
Liniara: daca |h(n) - h*(n)| <= const
Subexponentiala: |h(n) - h*(n)| <= O(log(h*(n)))
Exponentiala: tot mai buna decat cautarile neinformate
Algoritmi neinformati: BFS(V + E), DFS(V + E), Dijkstra(E * log V)
A* este informat, are functie euristica, stie la fiecare pas drumul pe care il are si acesta este ales pe baza euristicii (nu e luat random, sau “primul venit, primul servit”).

1. Construiti arborele Huffman de compresie pentru urmatorul text “ce abecedar” si calculati care este costul acestuia (cu 2 zecimale).
In caiet

3. Taieturile alfa-beta reduc complexitatea unei rezolvari cu minimax pentru jocuri doar daca sunt folosite impreuna cu o alta tehnica. Care este aceasta si de ce este importanta? Dati un scurt exemplu care demonstreaza ca folosirea acesteia impreuna cu alfa-beta aduce imbunatatiri.
In plus, pe langa taierile alfa-beta, conteaza ordinea in care se fac mutarile. Aceasta nu este costisitoare pe langa complexitatea exponentiala a algoritmului. Mai mult, se pot folosi euristici pentru a alege mutarile mai avantajoase primele. De exemplu in sah, daca am regina in pozitia de a lua atat nebunul, cat si pionul, trebuie sa alegem o tactica dupa care luam piesele (poate iau nebunul cu puncte mai multe, insa sunt atacata dupa de o alta piesa).

4. Daca pentru fiecare nod n din cadrul unei explorari folosind algoritmul A* euristica este definita in felul urmator: h(n) = max{cost(n, n’) | n’ $\in$ succs(n)} (costul muchiei de cost maxim care pleaca din n catre un succesor al sau). Ce puteti spune despre aceasta euristica si despre rezultatul intors de catre algoritm?
Cum A* cauta mereu drumul de cost minim, euristica folosita nu este un admisibila, deoarece alege mereu drumurile de cost maxim. Totusi, aceasta nu impiedica algoritmul sa gaseasca solutia corecta, ci doar scade optimalitatea.

5. Descrieti ce reprezinta o subproblema si determinati relatia de recurenta pentru problema parantezarii matricilor.
Subproblema - o parte din problema initiala care ajuta la rezolvarea acesteia; aceasta este rezolvata inaintede problema in sine si rezultatul ei este propagat din subproblema in subproblema pana la solutia finala
$A_{24} = A_2 * A_{34}$
$A_{24} = A_{23} * A_4$
+ formula generala
Este minimul dintre acestea. + exemplu

1. Care sunt pașii pe care trebuie să îi urmați pentru a rezolva o problemă prin programare dinamică? Exemplificați acești pași pentru problema parantezării matricilor studiată la curs, spunând înainte de toate cerința acestei probleme.
Avem un numar n de matrice si dorim sa inmultim aceste matrice facand un numar cat mai mic de operatii de inmultire. Pentru a ajunge la acest rezultat, trebuie sa grupam matricele si sa parantezam adecvat pentru a vedea care matrice inmultesc primele, care se inmultesc intre ele etc.
2. Care este numărul maxim, respectiv minim, de relaxări efectuate de către algoritmul Bellman-Ford pentru fiecare muchie a unui graf orientat? Dar pentru algoritmul lui Dijkstra?
Numarul maxim este V - 1, iar minimul este 1 (pentru costuri pozitive), sau nu se relaxeaza niciodata daca nu am drum din sursa catre nod.
Minimul pentru Dijkstra este 1 sau 0 (ca mai sus). Maxim la fel: exemplu caiet

3. Pornind de la definiție, explicați de ce este important ca euristica folosită de algoritmul A* să fie admisibilă? Demonstrați apoi că orice euristică consistentă este admisibilă.
Eursitica este admisibila daca pentru orice n din spatiul starilor, h(n) <= h*(n), ceea ce inseamna ca euristica este una optimista.
Daca este consistenta, inseamna ca pentru n’ succesor al lui n, h(n) <= h(n’) + k(n, n’).
Fie n’ = x - nod solutie => k(n, x) = min {k(n, y) | y - nod solutie} = h*(n)
h(n) <= h*(n) + h(x) si h(x) = 0 => h(n) <= h*(n) => euristica admisibila

4. Definiți ce înseamnă graful rezidual al unei rețele de transport și demonstrați (chiar și informal) că atunci când nu mai există nici o cale de ameliorare de la sursă la drenă în graful rezidual înseamnă că am descoperit tăietura minimală a rețelei de transport.
Definitie mai sus. Daca in graful rezidual nu mai avem nicio cale de la sursa la drena, atunci suntem siguri ca am descoperit toate caile posibile, deoarece acest graf ne spune si care sunt caile reziduale si ce cai directe mai exista intre noduri.

1. Să presupunem că pentru o problemă oarecare de optim (de ex. problema selecției activităților sau a programării spectacolelor) puteți găsi atât o rezolvare lacomă, cât și una prin programare dinamică, care să întoarcă soluția corectă.
a) Care dintre ele are (în general) o complexitate mai bună și de ce?
In general algoritmul greedy este mai rapid decat programarea dinamica, deoarece greedy considera mereu solutia care este buna in momentul curent si cu aceasta rezolva sub-problemele urmatoare, iar programarea dinamica consta in “consultarea” solutiilor anterioare si luarea unei decizii pe baza acestora (de aici timpul mai mare de rulare). Aceasta din urma retine in anumite cazuri si solutiile anterioare la fiecare pas, avand si o complexitate spatiala mai scazuta.
b) Ce proprietăți trebuie să aibă o problemă pentru a ne afla în situația descrisă mai sus (să accepte atât o soluție cu programare dinamică, cât și una greedy)?
Problema trebuie sa poata fi impartita in mai multe subprobleme(DP) si fiecare problema sa aiba solutie optim locala care sa fie mai apoi solutie optim globala(greedy). Greedy are o abordare top-down, deci in momentul in care problema poate fi rezolvata cu DP cu aceasta abordare si se respecta si prima conditie enuntata, atunci avem o problema ce poate fi rezolvata prin ambii algoritmi.

2. Care sunt cele 4 tipuri de muchii care pot apare într-un graf orientat în urma unei parcurgeri în adâncime? Explicați cum puteți identifica fiecare dintre aceste muchii în timpul parcurgerii în adâncime.
Arc direct - nod gri -> nod alb
Arc inainte - nod gri -> nod negru (cand timpul de debut al nodului gri este mai mic decat cel al nodului negru)
Arc invers - nod gri -> nod gri
Arc transversal - nod gri -> nod negru (cand timpul de debut al nodului gri este mai mare decat cel al nodului negru)

4. Explicați care sunt principalele două consecințe ale alegerilor random luate de către algoritmii aleatori pentru rezolvarea unei probleme oarecare. Apoi, specificați care dintre ele este valabilă pentru algoritmii de tip Las Vegas, respectiv Monte Carlo.
Principalele consecinte sunt gasirea unei singure solutii si determinarea solutiei optime.
Las Vegas se bazeaza pe determinarea solutiei optime, iar Monte Carlo pe gasirea unei singure solutii.

5. Asemănări și diferențe între algoritmii de tip backtracking si cei folosiți pentru propagarea constrângerilor. Cum pot fi combinați aceștia pentru rezolvarea unei probleme dificile?
O asemanare ar fi ca ambii folosesc recursivitatea. Propagarea restrictiilor nu ajunge la rezolvarea completa a problemei si de aceea depinde de backtracking care genereaza solutii partiale. Backtracking-ul are o complexitate exponentiala si are nevoie de o limitare a spatiului de cautare daca de propagarea restrictiilor pentru a fi mai eficient.

1. Un sistem de învățare automată își propune să reconstruiască la ieșire textul primit la intrare folosind diferite euristici. Pentru această sarcină, două euristici au fost propuse: una care determină mulțimea literelor din fiecare din cele două texte (de intrare și de ieșire) și determină câte litere are diferența acestor mulțimi și alta care calculează distanța de editare dintre cele două texte (numărul de operații necesare pentru a transforma un text în altul). Care dintre cele două euristici este mai informată? Ce înseamnă acest lucru?
A doua euristica este mai informata, deoarece spune exact cate modificari trebuie facute ca sa ajungi de la textul initial la cel final. Prima, in schimb, determina doar literel care sunt diferite, insa nu tine cont de pozitia acestora. De exemplu:
Luam cele doua texte “Nan” si “Nna”. Prima euristica va intoarce 0 (am aceleasi litere in cele doua texte), iar a doua euristica 2, deoarece doua poziiti nu contin literele bune.
A doua este mai informata, ceea ce inseamna ca ofera cat mai multe informatii si duce mai repede la rezultatul final corect.

3. Dându-se numai matricea rezultată în urma rulării algoritmului Floyd-Warshal pe un graf (nu avem acces nici la graf, nici la costurile arcelor), se poate detecta dacă graful respectiv a avut sau nu ciclu(ri) de cost negativ sau nu? Cum? Dacă a avut cicluri de cost negativ, se poate stabili ce noduri au făcut parte din cel puțin un astfel de ciclu de cost negativ? Cum?
Ca sa ne dam seama daca avem sau nu ciclu de cost negativ, ne uitam pe diagonala principala. Aceasta ar trebui sa contina doar 0-uri. Daca vreo valoare este diferita, inseamna ca avem cicul de cost negativ. Putem spune ce noduri fac parte din cel putin un ciclu daca acestea au pe diagonala lor valoare negativa.

5. Stabiliți valoarea de adevăr a propoziției următoare și argumentați răspunsul dat: Dacă în timpul rulării algoritmului DFS pe un graf orientat conex se găsește un arc invers, atunci rulând algoritmul DFS pe același graf plecând din orice alt nod va duce la găsirea cel puțin a unui arc invers.
Detectia de arc invers reprezinta existenta unui ciclu in graf. Daca am porni din orice alt nod, ciclul inca ar ramane si deci inca s-ar gasi cel putin un arc invers.

1. Care sunt pașii principali care trebuie urmați pentru a rezolva o problemă folosind programarea dinamică? Exemplificați pe scurt aplicarea acestor pași pe una dintre problemele discutate la curs.
O solutie optima contine solutiile optime ale subproblemelor sale.
Problema P se descompune recursiv in subprobleme care accepta solutii din ce in ce mai simple. La final are loc suprapunerea subproblemelor si aflarea solutiei optime a problemei initiale. Pe parcursul rezolvarii subproblemelor, rezultatele se retin in variabile/vectori/matrice pentru a nu se recalcula la pasii viitori.
O problema discutata la curs este determinarea distantelor minime intr-un graf de la toate nodurile la toate nodurile. Aceasta se bazeaza pe algoritmul Floyd-Warshall ce are in spate programarea dinamica. Acesta retine la fiecare pas o matrice cu costurile minime pana la momentul respectiv ce este folosita la pasul urmator unde este actualizata. (++ parantezarea matricelor)

3. Din ce cauză este necesar ca algoritmul A* să permită scoaterea nodurilor din mulțimea CLOSED și readăugarea acestora în mulțimea OPEN? Când se întâmplă acest lucru și ce asigură el? Știți vreo situație în care se poate garanta că nici un nod nu va fi scos din CLOSED pentru a fi reintrodus în OPEN?
Algoritmul trebuie sa permita scoaterea din CLOSED, deoarece sunt situatii cand unor noduri le sunt actualizate distantele minime (g) si acesta trebuie sa fie reverificat. Daca am avea o euristica suficient de admisibila care sa gaseasca de la inceput drumul optim dintre doua noduri, atunci nu ar fi nevoie de actualizarile distantelor pentru unele noduri.

4. Ce este o muchie înapoi (ce culori au vârfurile ce determină muchia respectivă)? Demonstrați că un graf aciclic nu are voie să conțină muchii înapoi în parcurgerea în adâncime.
Nodurile ce formeaza muchia inapoi au ambele culoarea gri.
Presupunem ca ar avea o muchie inapoi. Exista doua noduri u si v ce au muchie intre ele (u -> v). Ca aceasta sa fie muchie inapoi, inseamna ca atat u, cat si v trebuie sa fie gri si d(v) < d(u). Dar u -> v => d(u) < d(v) sau avem ciclu contradictie. Deci nu se poate sa avem muchie inapoi intr-un graf aciclic.

5. Schițați, pe scurt, pseudocodul pentru algoritmul minimax. Spuneți apoi minim 2 posibilități prin care acesta poate fi îmbunătățit pentru a reduce timpul de rezolvare.
Nivel MAX: jucatorul intoarce max{valorile de pe nivel}, apel recursiv pentru jucator MIN si nivel + 1
Nivel MIN: jucatorul intoarce min{valorile de pe nivel}, apel recursiv pentru jucator MAX si nivel + 1
Sortarea mutarilor, taieturile alfa-beta

1. Cum se poate modifica algoritmul lui Dijkstra daca graful este aciclic si orientat?
Daca am avea costuri pozitive, atunci Dijkstra ar functiona corect, iar daca am avea costuri negative, atunci am aplc Bellman-Ford optimizat (aia cu marcat).

5. Ce inseamna ca o euristica pentru algoritmul A* este admisibila? Ce s-ar intampla, daca aceasta conditie nu s-ar respecta?
Daca nu se respecta, atunci solutia este gasita intr-un timp mai mare de rulare.

2. Explicați dacă/când puteți rezolva o problemă dificilă (de ex. Sudoku) folosind doar un algoritm pentru satisfacerea constrângerilor. Apoi spuneți care este îmbunătățirea pe care algoritmul AC3 o aduce în plus față de AC1. Există dezavantaje pentru utilizarea AC3?
??

4. Explicați în propriile cuvinte sau demonstrați folosind proprietățile/teoremele prezentate la curs de ce algoritmul Bellman-Ford are nevoie de un număr finit de relaxări de muchii. Care este acest număr și care este ordinea în care sunt relaxate muchiile (dacă există o astfel de ordine)?
Bellman-Ford bazandu-se pe programarea dinamica, abordarea pe care o adopta este din aproape in aproape, folosind la fiecare pas ceea ce a calculat anterior. La prima iteratie calculeaza distantele cele mai scurte pentru care au o singura muchie in drumul minim final, apoi gasesc drumurile minime care contin doua muchii (si stiu ca acest drum este si cel minim). Pot avea maxim |V| - 1 muchii intre doua noduri (presupunand ca distanta minima dintre acestea este daca trec prin toate celelalte noduri) si deci am nevoie de |V| - 1 iteratii ca sa fiu sigura ca traversez toate muchiile si gasesc drumul minim. Nu conteaza ordinea.

5. Cum se definește complexitatea pentru un algoritm aleator de tip Monte-Carlo? Explicați în propriile cuvinte care credeți că este utilitatea acestor algoritmi (eventual folosind un exemplu).
f(n) = O(g(n)) => En_0 > 0, c > 0 a.i Vn >= n_0, 0 <= f(n) <= c * alfa * g(n), alfa > 0, afla fixat si mare, avand probabilitatea cel putin 1 - n^{-alfa}(probabilitate mare).

3. Care este numărul minim, respectiv maxim, de componente tare conexe ale unui graf care admite o sortare topologică? Justificați!
Ca sa putem sorta topologic un graf, ar trebui sa nu existe niciun ciclu. Atat numarul minim, cat si numarul maxim ar fi n - numarul de noduri, in momentul in care fiecare nod este o componenta separata.

1. Dându-se o matrice A cu n linii și n coloane, descrieți o modalitate eficientă pentru a calcula A la puterea n (A^n). Care este complexitatea acestui algoritm și ce tehnică folosește?
Se foloseste metoda Divide et Impera, impartindu-se la fiecare apel recursiv numarul n (puterea) la doi. Astfel, calculam $A^{n/2} * A^{n/2}$ daca n este par sau $A^{(n - 1) / 2} * A^{(n - 1) / 2} * A$.
Complexitatea este O(log n).

2. Comparație între BFS și algoritmul lui Dijkstra.

BFS	Dijkstra
Parcurge nodurile nivel cu nivel de la cele care sunt cele mai aproape de sursa	La fiecare pas, ia nodul cu costul cel mai mic
Poate fi folosit pe grafuri ce nu au costuri pe muchii sau pe cele care au un cost egal	Merge atat pe grafuri cu costuri pe muchii (pe costuri pozitive), cat si pe cele care nu au
Foloseste coada ca structura de date	Foloseste coada de prioritati
O(V + E)	O(ElogV)

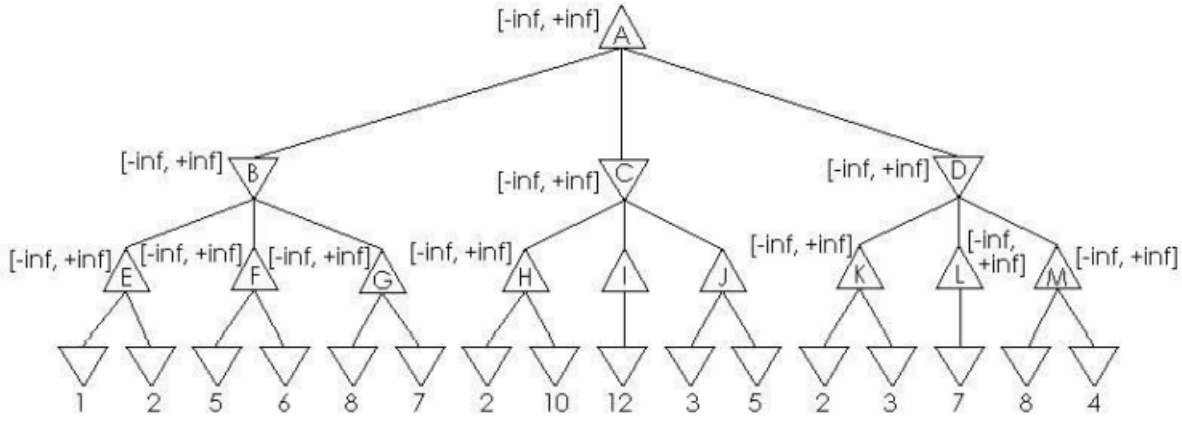
3. Descrieți pe scurt ideea din spatele algoritmului lui Johnson și calculați care este complexitatea fiecărui pas, plus complexitatea totală.
Daca ar fi nevoie de aplicarea lui Bellman-Ford, aici complexitatea este O(V * E)
Calculul potentialelor: O(V)
Recalcularea costurilor de pe muchii: O(E)
Dupa care se aplica Dijkstra de V ori => O(V * Vlog V + V*E) / O(EVlogV)
Deci complexitatea totala: $O(V^2 * log V + 2 * V * E) = O(V^2 * log V + V * E)$

5. Ce este tăietura minimală și care este legătura dintre ea și fluxul maxim (maxim 2 fraze). Cum se poate deduce tăietura minimală pentru un graf de transport?
Taietura minimala ne spune cand un flux este maxim. Aceasta se poate deduce daca f = c.

2. Explicați modul în care algoritmul lui Kosaraju determină componentele tare conexe într-un graf orientat, insistând asupra rolului fiecărei dintre cele două parcurgeri în adâncime efectuate de către acesta. De ce este importantă ordinea alegerii nodurilor în cea de-a doua parcurgere?
Aplicarea pe un DFS normal ar duce la suprapunerea unor CTC-uri (sa am noduri dintr-o CTC intr-alta).

4. Comparați algoritmii de tip greedy cu cei de programare dinamica. Dați 3 exemple din fiecare categorie.
Greedy: Dijkstra, Prim, Kruskal
DP: Floyd-Warshall, Bellman-Ford, Chain Matrix Multiplication

5. Aplicați algoritmul alfa-beta pe arborele de mai jos, analizând nodurile de la stânga spre dreapta. Se cer: identificarea valorii nodului A si evidențierea nodurilor tăiate de către algoritm.



Taiate: F6, G7, M4, DL, DM
E = 2; F = 5; G = 8
H = 10; I = 12; J = 5
K = 3; L = 7; M = 8
B = 2; C = 5; D = 3
A = 5

4. Care este numărul minim și maxim de punți și puncte de articulație ale unui graf neorientat cu n noduri si a arce? Justificați.
Minim puncte = 0 (graf = ciclu)
Maxim puncte = n - 2 (graf = lant)
Minim punti = 0 (graf = ciclu)
Maxim punti = a (graf = lant)

3.[4p] Comparați algoritmul de construcție a Arborilor Huffman cu cel pentru construcția Arborilor Optimi la Căutare.
b. Se cunoaște probabilitatea căutării unei chei care este în arbore
c. Sunt arbori binari de căutare
d. Se cunoaște probabilitatea căutării unei chei care nu este în arbore
b. Mulțimea cheilor din arbore trebuie cunoscută în avans
c. Arborele optimal de căutare rezultat poate să nu fie echilibrat în anumite cazuri

4.[4p] Demonstrați că o euristică este consistentă dacă și numai dacă este monotonă.

1.[4p] Care este numărul maxim de înălțări în algoritmul preflux? Argumentați.
Inaltimea maxima: 2V - 1
Nr maxim de inaltari: (2V - 1)(V - 2)
Pompari saturate: 2VE
Pompari nesaturate: $4V^2(V + E)$

4.[4p] Care sunt metodele de îmbunătățire ale backtrackingului cronologic? Hint: nu este vorba de AC1, AC-3, etc., aceștia fiind algoritmi, nu metode.
Sa faca o ordonare initiala a nodurilor dupa niste criterii pentru a face mai putine reveniri.
Sa ia deciziile doar dupa ce analizeaza efectul lor in viitor.

1. Enunțați și schițați demonstrația teoremei căii albe într-un graf.
G = (V, E) si exista un Arb(u). Nodul v este descendent al lui u ⇔ in momentul d(u) - cand se descopera u - exista un drum plin de noduri albe de la u la v.

3. Comparați schemele de algoritmi Greedy și Divide et Impera.

Greedy	Divide et Impera
Porneste de la o solutie de optim local si rezolva dupa celelalte subprobleme	Descompune problema in subprobleme si recursiv imbina solutiile pentru a ajunge la solutia finala
Odata ce a trecut de un pas, nu mai revine	Poate calcula un pas de mai multe ori
Are un citeriu de optim	Nu are criteriu de optim

4. Numarul minim de sortari topologice: 0
Numarul maxim: n!